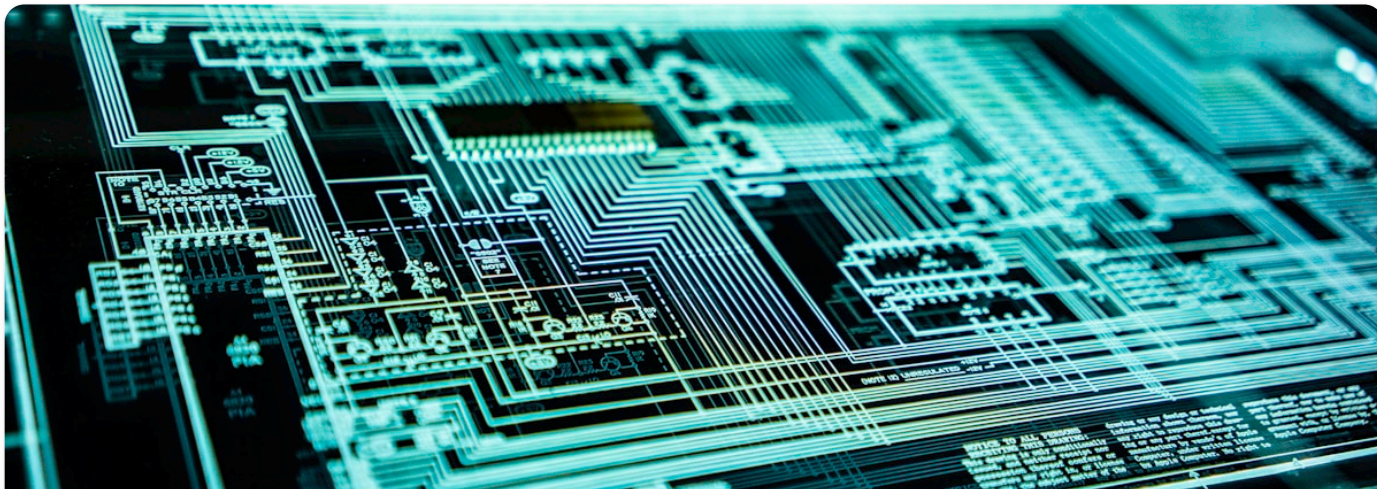


言零的博客 - 20260405



漏洞分析

虚拟发卡系统回调伪造漏洞全链路拆解 — 从备份文件泄露到批量盗取卡密

深度分析一起针对 ACG-Faka + BEpusdt 的真实攻击事件：攻击者通过两条路径窃取支付密钥——下载 Web 根目录残留的备份文件，或利用 Nginx 配置错误直读 Config.php 源码——进而伪造合法回调通知批量盗取卡密。附官方安全公告、五站实测审计结果与完整防御方案。



Thursday, August 13, 2026



5 minutes read

> TABLE OF CONTENTS

一分钟速读：一个朋友的虚拟发卡在在一夜之间被批量盗走了上百张卡密。攻击者没有碰数据库，没有拿 Shell，甚至没有登录后台——他只是向一个公开的回调接口发送了一条精心构造的"支付成功"通知，系统就乖乖地把卡密吐了出来。本文从攻击日志逆向还原完整攻击链，揭示密钥泄露的两条路径——**备份文件暴露**（官方公告确认）与 **Nginx 配置错误导致 Config.php 源码可读**（日志分析发现），拆解回调签名机制的致命弱点，并用同一套方法对五个同类站点进行实测审计。

前言

事发后，ACG-Faka 官方发布了安全公告：

严禁在网站根目录存放备份文件。 近期排查发现，多个 V3 新版网站出现伪造支付回调，主要原因是在网站根目录存放了整站备份、数据库备份或 ZIP 压缩包。这些文件可被公网直接下载，其中通常包含支付密钥、货源接口密钥、数据库账号密码等敏感信息。攻击者获取后，可直接伪造支付回调并自动提取商品。

公告一针见血：**问题不在签名算法，而在密钥泄露。** 签名算法再安全，密钥被攻击者拿到了也白搭。

那密钥是怎么泄露的？公告指出是备份文件，我们的日志分析还发现了另一条独立路径——Nginx 配置错误导致 PHP 源码直接暴露。两条路径殊途同归，都指向同一个运维盲区：**敏感**

文件可被公网直接下载。

虚拟发卡系统的核心流程很简单：

用户下单 → 跳转支付网关 → 链上确认 → 网关回调通知 → 系统标记已付 → 释放卡密

问题出在第四步。支付网关（BEpusdt）向发卡系统（ACG-Faka）发送回调通知时，系统需要验证这条通知**确实来自网关、而非攻击者伪造**。如果验证机制存在缺陷，攻击者就可以跳过真实支付，直接"通知"系统订单已付，白拿卡密。

这不是假设场景。我拿到了一份真实攻击日志，下面是完整拆解。

一、攻击复盘：从日志还原 660 条请求

1.1 攻击时间线

通过分析受害站点的 Nginx Access Log，攻击者（IP: 203.27.106.***）的行为可以清晰地分为四个阶段：

时间	阶段	行为	日志特征
14:22	路径探测	尝试 /callback/BEpusdt vs /callback.BEpusdt	404 → 200，锁定正确路径格式
14:50	签名试探	发送全零签名 000...000	29 字节响应 = {"code":0,"msg":"sign error"}
17:01	源码窃取	反复请求 /app/Pay/BEpusdt/Config/Config.php	200，响应体包含 PHP 源码

时间	阶段	行为	日志特征
17:32	首次得手	发送携带正确签名的伪造回调	33 字节响应 = <code>{"code":200,"msg":"success"}</code>
18:31+	批量收割	自动化脚本，每 6 秒一单	持续 33 字节成功响应

注意 14:50 到 17:01 之间的两小时空白——攻击者在第一次签名试探失败后，**没有继续暴力枚举签名**，而是转向了一个完全不同的方向：读源码。

1.2 响应大小指纹

Nginx 日志中的 `body_bytes_sent` 字段暴露了一切：

29 字节 → {"code":0,"msg":"sign error"} # 签名错误, 攻击失败
35 字节 → {"code":0,"msg":"handle not found"} # 插件不存在
33 字节 → {"code":200,"msg":"success"} # 攻击成功, 订单被标记已付

当日志中突然出现 33 字节的响应时, 攻击已经得手了。

二、回调签名机制：为什么能被伪造

2.1 BEpusdt 的签名算法

BEpusdt 是一个 USDT 收款网关, 当链上确认收款后, 它向商户系统发送 HTTP POST 回调。

签名计算方式:

signature = MD5(按字母排序的参数字符串 + API_TOKEN)

具体来说：

1. 取所有非空参数 (排除 signature 本身)

```
params = {k: v for k, v in data.items() if v != '' and k != 'signatu
```

2. 按 key 字母排序, 拼接成 key=value& 格式

```
sorted_str = '&'.join(f'{k}={params[k]}' for k in sorted(params.keys
```

3. 末尾拼接 API Token, 取 MD5

```
signature = md5(sorted_str + api_token)
```

商户端验证时, 用自己存储的 API Token 重新计算签名, 与收到的签名做比对。

这个算法本身没有问题。问题在于：

1. 如果 API Token 泄露了，攻击者可以算出合法签名
2. 如果 API Token 为空，签名 = MD5(params + "")，任何人都能算
3. 如果比对方式有缺陷（PHP 弱类型），签名验证可被绕过

2.2 ACG-Faka 的回调处理流程

ACG-Faka 收到回调请求后的处理链：

`/user/api/order/callback.{Handle}`

↓

路由到对应插件 (BEpusdt / Epusdt / Alipay)

↓

插件的 `Signature.php` 验证签名

↓

验证通过 → `Order.php` 更新订单状态 → 释放卡密

↓

返回 "success" (BEpusdt) 或 "ok" (Epusdt)

关键文件：

文件	作用
app/Pay/BEpusdt/Config/Config.php	存储 API Token（签名密钥）
app/Pay/BEpusdt/Impl/Signature.php	签名验证逻辑
app/Service/Bind/Order.php	订单状态更新

三、根因分析：密钥泄露的两条路径 + 两重代码缺陷

攻击成功的前提是拿到 API Token。从目前掌握的信息看，攻击者有两条互相独立的路径可以拿到密钥，命中任何一条就够了。

3.1 路径一：备份文件泄露（官方公告确认）

这是官方通告明确指出的主因。很多站长习惯用宝塔面板的"一键备份"功能，把整站打包成 .zip 或 .tar.gz 放在网站根目录：

```
/www/wwwroot/faka/  
├─ index.php  
├─ app/  
│   └─ Pay/BEpusdt/Config/Config.php    ← 包含 API Token  
├─ config/  
│   └─ database.php                      ← 包含数据库密码  
├─ site_backup_20260810.zip              ← 整站备份，包含以上所有文件  
└─ db_backup_20260810.sql                ← 数据库导出，包含所有订单和卡密
```

Nginx 默认会把 .zip 当静态文件返回。攻击者只需猜到文件名（常见模式：

backup.zip、www.zip、域名.zip、日期.zip），就能一次性下载全站源码 + 数据库，其中包括支付密钥、数据库凭据、货源接口密钥——全部明文。

这比后面要说的 Config.php 泄露更严重，因为数据库备份里连卡密都有，攻击者甚至不需要走伪造回调这条弯路。

3.2 路径二：Nginx 配置错误导致 Config.php 直接暴露

这是我们在日志分析中发现的另一条路径。ACG-Faka 的 PHP 配置文件存放在 /app/Pay/BEpusdt/Config/Config.php 。如果 Nginx 没有做目录级 deny：

```
# ❌ 缺少这条规则，攻击者可以直接请求 PHP 文件
location ~* ^/(app|config|runtime|vendor)/ {
    deny all;
}
```

后果因 Nginx 对 .php 的路由配置而异：

Nginx 配置	请求 Config.php 的结果	危险等级
.php 正确路由到 PHP-FPM	PHP 执行 return [...]，输出为空 (200 空响应)	⚠ 中
.php 未路由到 PHP-FPM（当静态文件返回）	PHP 源码原文输出，API Token 明文暴露	🔴 致命

第二种情况在宝塔面板上比想象中常见——如果 Nginx 的 `location ~ /\.php$` 规则只匹配了网站根目录下的 `.php` 文件，深层路径（如 `/app/Pay/.../Config.php`）的 `.php` 可能不会交给 PHP-FPM 处理，而是被当成普通文本文件返回。

攻击者直接看到了：

```
<?php
return [
```

```
'api_url' => 'https://pay.example.com',  
'api_token' => 'a1b2c3d4e5f6...', // ← 签名密钥, 明文暴露  
];
```

我们审计的五个站中, Config.php 都返回 200 空响应 (PHP 正确执行了 `return []`), Token 没有泄露。但路径可达本身就是隐患——如果未来某次运维操作改了 PHP-FPM 配置, Token 就会暴露。

两条路径的关系: 备份文件是“一锅端”, Config.php 是“精准窃取”。攻击者会两条路都试——先扫备份文件 (自动化成本低), 扫不到再尝试直接读 Config.php。实际攻击日志中两种行为都有出现。

3.3 代码缺陷一: PHP 弱类型比较 (高危)

即使 Token 没有泄露, BEpusdt 插件的签名比对方式也可能存在问题。如果 Signature.php 使用了 PHP 的 `==` (宽松比较) 而非 `===` (严格比较) 或 `hash_equals()`:

// ❌ 危险：PHP 弱类型比较

```
if ($calculated_sign == $received_sign) { ... }
```

// 攻击向量：

// - signature=0 → int(0) == "any_string" 在特定条件下为 true

// - signature=false → false == "" 为 true

// - signature=[] → 数组与字符串比较行为不可预测

安全的实现应该是：

// ✅ 安全：类型检查 + 时序安全比较

```
if (!is_string($sign) || $sign === '') {
```

```
    return false;
```

```
}
```

```
return hash_equals($calculated, $sign);
```

hash_equals() 的作用有两个：

1. 防止类型混淆——只接受字符串
2. 防止时序攻击——比较耗时恒定，不会因为前几位匹配而提前返回

3.4 代码缺陷二：回调端点无访问控制（中危）

回调端点 `/user/api/order/callback.BEpusdt` 对任何 IP 开放。攻击者可以从任意位置发送请求，系统只靠签名验证来区分合法回调与伪造请求。

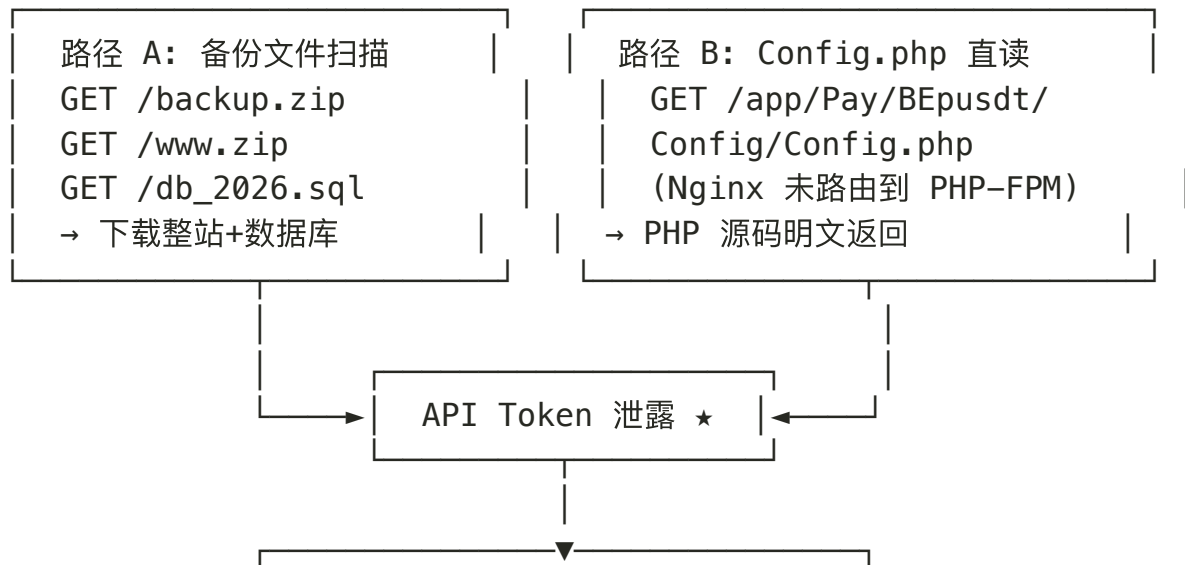
理想状态下，应该叠加 IP 白名单：

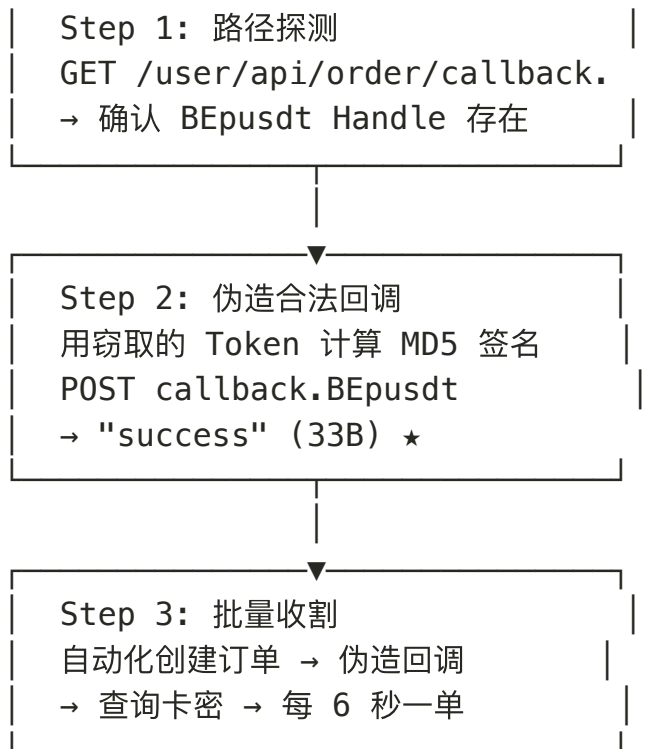
```
location ~* ^/user/api/order/callback\. {  
    allow 1.2.3.4;    # 支付网关服务器 IP  
    deny all;  
}
```

即使签名验证被绕过，IP 白名单也能兜底。

四、完整攻击链

两条密钥泄露路径最终汇聚到同一个攻击终点：





整个过程不需要任何漏洞利用工具，只需要 `curl` 和 `md5sum`。路径 A 甚至不需要走回调——数据库备份里直接就有卡密。

五、五站实测审计

拿到攻击手法后，我用同一套方法对五个同类站点进行了黑盒审计。测试矩阵包含 13+ 种攻击向量，每次攻击后通过 API 验证订单状态是否变化。

5.1 攻击向量

#	向量	目标
A1	全零签名 / 随机签名	基础验签能力
A2	空签名 / 缺失签名字段	空值绕过

#	向量	目标
A3	signature=0 (int)	PHP == 弱类型: 0 == "string"
A4	signature=false	PHP == 弱类型: false == ""
A5	signature=[] (数组)	PHP 数组比较行为异常
A6	空密钥 MD5 签名	Token 为空时可伪造
A7	空密钥 HMAC-SHA256	同上, 适配 Epusdt 插件
A8	金额篡改 (0.01)	金额绑定验证
A9	非成功 status 值	状态字段校验
A10	form-urlencoded 格式	Content-Type 混淆
A11	路径穿越 Handle	插件路由安全

5.2 审计结果

站点	框架	回调安全	状态验证	源码泄露	CF 防护	综合
站点 A	ACG-Faka	✅ 34/34 SAFE	有 API	路径泄露	无	⚠️ 中
站点 B	ACG-Faka	✅ 全部 SAFE	有 API	路径泄露	无	⚠️ 中
站点 C	Next.js/Vercel	N/A	需认证	无	全站	✅ 低
站点 D	ACG-Faka	✅ 全部 SAFE	无 (404)	路径+composer	无	⚠️ 中
站点 E	ACG-Faka	✅ 64/64 SAFE	有 API	路径+composer	无	⚠️ 中

关键发现：

- 回调签名验证全部正常——五个站都没有被伪造回调突破
- 但四个 ACG-Faka 站都有源码路径泄露——攻击者可以看到 Fatal Error 中暴露的物理路径
- Config.php 虽然返回 200 空（PHP 正确执行了），但路径可达本身就是隐患

- **唯一一个 Next.js + Vercel 站完全不受此类攻击影响——非 PHP 框架，无弱类型问题，Vercel 不暴露源码**

5.3 为什么这五个站没被攻破

1. **没有备份文件暴露——五个站的 Web 根目录下没有扫到可下载的 .zip / .sql / .bak 文件**
2. **Config.php 正确走了 PHP-FPM 路由——返回 200 空响应，Token 没有泄露**

但隐患仍在：四个 ACG-Faka 站的 /app/ 路径仍然可达，只是 PHP-FPM 兜住了。如果未来某次运维操作改错了 PHP-FPM 配置，或者站长在根目录放了一次备份文件，攻击者立刻就能得手。这是一颗定时炸弹。

六、防御方案

6.1 P0 紧急修复（立即执行）

清理并封锁备份文件——这是最紧急的一步：

1. 立即删除 Web 根目录下的所有备份文件

```
find /www/wwwroot/faka/ -maxdepth 1 \  
  \( -name "*.zip" -o -name "*.tar.gz" -o -name "*.rar" \  
    -o -name "*.sql" -o -name "*.bak" -o -name "*.7z" \) \  
  -exec rm -f {} \;
```

2. 在 Nginx 中彻底封锁备份文件下载

封锁所有备份/归档文件类型

```
location ~* \.(zip|tar\.gz|rar|sql|bak|7z|gz|tgz)$ {  
    deny all;  
    return 403;  
}
```

Nginx 封锁源码目录——阻断 Config.php 泄露路径：

```
# 阻止所有对 PHP 源码、配置、运行时文件的直接访问
location ~* ^/(app|config|runtime|vendor)/ {
    deny all;
    return 403;
}

# 阻止敏感文件
location ~* (composer\.(json|lock)|\.env|\.git) {
    deny all;
    return 403;
}
```

[Copy](#)

更换所有凭据——如果曾经在 Web 目录放过备份文件，必须假设已泄露，一律重置：

- API Token（支付密钥）
- 数据库密码

- 货源接口密钥
- 后台管理员密码

升级签名验证——确认 Signature.php 使用 hash_equals() + is_string() 类型检查:

```
public static function safetyEquals(mixed $str, string $local): bool
{
    if (!is_string($str) || $str === '') {
        return false;
    }
    return hash_equals($local, $str);
}
```

6.2 P1 加固措施

回调端点 IP 白名单:

```
location ~* ^/user/api/order/callback\. {  
    allow 支付网关IP;  
    deny all;  
}
```

签名算法升级——从 MD5 升级到 HMAC-SHA256:

```
// MD5 拼接 (BEpusdt 默认) → 容易被长度扩展攻击  
$sign = md5($sorted_string . $token);
```

```
// HMAC-SHA256 (推荐) → 密钥参与哈希内部运算, 无法伪造  
$sign = hash_hmac('sha256', $sorted_string, $token);
```

6.3 P2 监控告警

- 回调端点频率限制 (单 IP 每分钟 ≤ 10 次)

- 异常响应监控：连续出现 33 字节（success）响应时触发告警
- 回调日志独立记录：IP、签名、时间、订单号，便于事后追溯

6.4 架构层面

如果条件允许，考虑迁移到非 PHP 框架（如 Next.js + Vercel），从根本上消除 PHP 弱类型比较和源码泄露的风险。审计中唯一零风险的站点正是这种架构。

七、总结

这次攻击的本质不是密码学层面的签名绕过，而是**运维层面的密钥泄露**。攻击者有两条路可走：

路径 A: Web 根目录残留备份文件 (.zip / .sql)

→ 攻击者下载备份 → 解压得到全部凭据 (甚至直接拿到卡密)

路径 B: Nginx 配置错误, Config.php 当静态文件返回

→ 攻击者读取 PHP 源码 → 得到 API Token

两条路殊途同归:

→ 用 Token 计算合法 MD5 签名

→ 伪造回调通知

→ 系统认为已付款

→ 卡密被释放

官方通告指向路径 A, 日志分析发现路径 B。多个受害站点同时存在两个问题。

修复其实不复杂——删备份文件、加 Nginx deny 规则、换密钥, 三步就能阻断整条链。但现实是, 四个受审计的 ACG-Faka 站点连 /app/ 目录的 deny 规则都没有。

安全不是只靠签名算法。**签名验证是最后一道防线，不应该是唯一一道。**纵深防御意味着即使签名被绕过，IP 白名单能兜底；即使白名单没配，源码和备份至少不应该暴露密钥。任何一层失守都不会导致全盘崩溃——这才是安全架构的正确姿势。

本文所有测试均在授权范围内进行，已获得站点所有者许可。文中不包含任何真实域名和凭据。

站长推荐

SwitchBase — 站长自用 API 中转站

强推 Claude 全系列，纯血自有 CCMAX 号池，在线率 99%+。不算最便宜，但稳定性一骑绝尘，站长本人长期使用中。

[前往注册 →](#)

推荐服务

Florence AI — AI 会员一站式充值

ChatGPT Plus / Pro · Claude Pro · Grok — 正规信用卡渠道，自动发货 30 秒到账

[前往充值 →](#)

[支付安全](#)

[Web 安全](#)

[PHP](#)

[回调伪造](#)

[ACG-Faka](#)

RELATED CONTENT

0 PHP 白嫖 ChatGPT Plus
— 跨区定价混淆攻击的完整技术拆解

焚决 Claude — 一个油猴脚本如何撬开 Anthropic 的支付大门

一次 Nginx Lua 木马响应实录：从移动站博彩站到 root 级入侵

© 2026 言零的博客 - 20260405

Built with Hugo

Theme **Stack** designed by Jimmy

站长推荐

API 中转站

推荐

AI 会员充值

Claude 全系列 · 自有号池
在线率 99%+ · 站长自用

switchbase.vip →

ChatGPT · Claude · Grok
正规渠道 · 自动发货 30s

faka.redeemai.me →